

20 Years of Software Engineering with Malte Ubl

Learnings from working eleven years working at Google, what tenure teaches you, and the similarities between framework and product teams.



GERGELY OROSZ

JUN 14, 2022 • PAID



43



4



Share



👋 Hi, this is [Gergely](#) with a subscriber-only issue of the Pragmatic Engineer Newsletter. If you're not a full subscriber yet, you missed the second part of [Inside Facebook's engineering culture](#) and the one [The tech boom potentially being over](#) and a look at [what companies are still 'business as usual'](#) in this turbulent market. Subscribe to get this newsletter every week 📌

For this edition, I caught up with a very impactful software engineer. It's no exaggeration to say his work directly affects the online experience of billions of people – right down to how fast a large number of webpages take to load.

[Malte Ubl](#) has been a software engineer for more than 20 years, He [spent 11 years at Google](#), starting as a Software Engineer 2 (L4), was promoted several times, and then left Google at the Principal Engineer level (L8).

At Google, Malte built a framework called [Wiz](#) that powers Search, Gmail, Photos, Drive, Meet, and other consumer UIs. He was central to creating AMP, and then applied the lessons from it to the Page Experience ranking system based on [Core Web Vitals](#), helping to make the web faster in the process.

A notable announcement he made was that Google Search was to stop supporting Microsoft's legacy web browser, Internet Explorer 11, late last year. This announcement caused a ripple effect, with other companies quickly following suit.



Malte Ubl
@cramforce

As a web developer this is one of the happiest announcements in a while: Google Search ended support for IE11 in its main product 🎉 (you can still search but will get a fallback experience). I'm mostly posting this so you can send it to your boss 😊 . We did the Math. It is time.

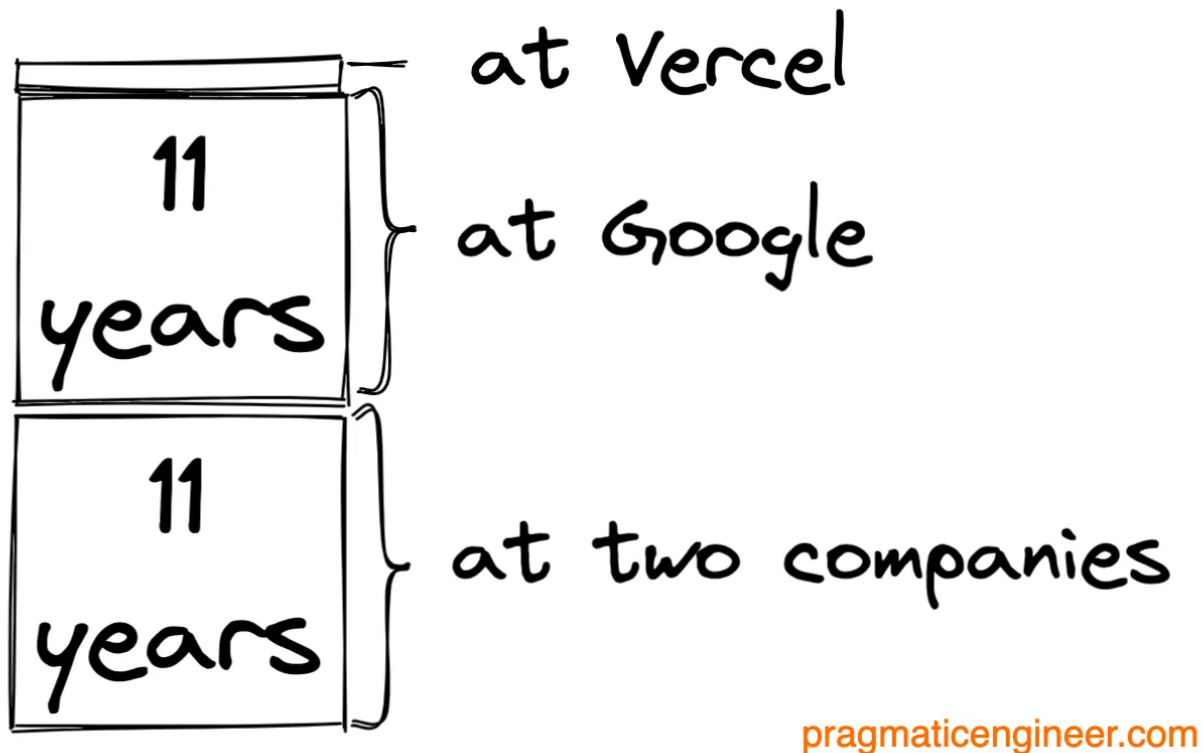
3:34 PM · Oct 1, 2021

4,914 Likes 1,076 Retweets

Malte recently left Google and joined [Vercel](#), a platform which enables frontend teams to work with an improved and smooth workflow.

I grabbed the opportunity to catch up with him and talk about the learnings from his career, what he found surprising about working at Google, his thoughts on working on frameworks, why he decided to leave Google for a fast-growing scaleup and what's different about working at Vercel compared to Google.

Visualizing Malte's career so far, this is how it would look like:



Malte's career, professional career, visualized

In this issue, we cover:

1. **Quick questions.** Including what is the best thing that happened in software engineering since Malte entered the software industry.
2. **Eleven years at two companies.** Learning from this time.
3. **From a small company to Google.** What was a thing where Google stood different from the previous places?
4. **Eleven years at Google.** On promotions, career progression and what tenure teaches you.
5. **Infrastructure vs product engineering.** What overlaps do they have?
6. **From Google to Vercel.** Why make the move, what's exciting about Edge computing and what was surprising to observe at Vercel?

1. Quick questions

I asked Malte three quick questions to warm things up.

1. What is your favorite programming language and why?

'Smalltalk. It's the first language I learned well, and I still think the language has good ideas. I picked it up when working an after-school job in a small software shop, when I was 14 years old in the mid '90s.

'Even today, I'd recommend the language as one to start out with. It's a good environment to explore things interactively and to get quick feedback. You don't have these 'traditional' cycles where you write code, then compile it, then run it, and then if it doesn't work you add logging statements.

'The language has disappeared over time, though. The company developing the language was good at engineering, but bad at marketing. Sun beat them with marketing and Java took their market.'

2. What is a hard problem in software engineering that is still not solved?

'Automated testing, especially for higher levels than unit testing. If you need to test anything that goes beyond a very well defined boundary, you'll get frustrated.

'For example, whenever I am working on a backend, this problem might come up as it might need an end-to-end test, or I might want to completely fake it out, or mock it out. Doing so is never straightforward. I've never seen creating non-trivial automated tests make anyone happy.

3. What do you think is the best thing to happen in software engineering since you entered the industry?

'Twitter, meetups, and conferences. It was around 2009 — 13 years ago — when the JavaScript community became a thing. Before that time, people wrote JavaScript in isolation. However, off the tail end of the AJAX revolution, JavaScript started to gain momentum, and a community started to form around it.

'Back then, I wanted to do a meetup for JavaScript developers. But then I had the problem of how can I find anyone in my city interested in the same problem space? 15

years ago, there was no good way to find these people.

'This was the time when Twitter was becoming popular, and it helped create this early JavaScript community. People could discover like minded others based on professional interests. Twitter made all of this not just possible, but also easy.'

2. Eleven years at two companies

'I spent the first nine years of my professional career at one company in Hamburg, Germany. This might sound like a long time, but, as context, this was a more common model in Germany. I went to a private university and the company paid my tuition. In return, they expected me to stay for a few more years after graduating.

'I started working at this company called Schaffhausen ITC Solutions when I was 18, and still in high school. I worked throughout my four years of college, and then four more years after that.

'One of my biggest regrets is that I never had time off work. I've been basically continuously employed since aged 14; working part-time through school, and with this first job since college.

'Schaffhausen ITC Solutions was an agency. I was the second employee in their web unit. This group grew to about 30 people by the time I left.

'Something I learned there which helped my career later on, was sales. I was not a salesperson, but it was a small enough company that I had to participate in this process. This is one advantage smaller companies have, versus ones with hundreds of people.

'At smaller companies, you get to play different roles and try out different things. You learn things quicker than if you're working in a corporation with a very well-defined role, and one that moves slowly.

'Seven years later, I decided it was time to move companies. I wanted to do more engineering work and joined this digital agency called SinnerSchrader. This was in 2009, when the JavaScript community was taking off. I managed to convince my

manager that we needed to fly to Washington DC to attend the first ever JSConf. At the event, I met another person from Germany and we decided we'd organize our own conference in Berlin.

'The JSConf in Washington DC was during the spring of 2009 and we organized the first JSConf in Berlin in November 2009. I was then involved with organizing all of the JSConf's until the last one which was in 2019.

'I became very involved with the JavaScript scene through JSConf. By the time I moved to San Francisco, I knew more people there through these events, than I did in Hamburg, Germany.

'One thing I learned from organizing conferences is working under a stressful timeline. A conference has a fixed "ship" date, and you have to ship whatever you have on that date, whether you are ready or not.

'However, organizing conferences also gives a sense of iteration. Every year, you get to start from scratch and redo how you organize the conference. We talk a lot about iterating in software and doing rewrites. With a conference, you actually get to do all of this.'

On becoming a manager

'I became a manager at my first employer, and ended up managing the whole web unit. Becoming a manager was basically trial-and-error. One day I was told I'd become a manager. I now joke that my entire career has been a seven-year oscillation between management and IC (individual contributor). I always saw management as a necessary evil — as in someone needs to do that job — more than a passion.

'My view on management is that moving into management should never be bundled with a promotion. Becoming a manager should always be a sideways move. This is a benefit of larger companies; once people move into a management role, if it doesn't work out, they have the flexibility to move back to being an IC.

'For small companies, this can be challenging, as there's often a pressure for you to keep doing the manager job, even if this position does not sit well with you.'

3. From a small company to Google

'I joined Google in Hamburg, Germany in 2010. I talked with Google a few times before, and also failed interviews once before. The second time, I got the job. I was excited to start, but my start did not go as I'd planned.

'On my first very first day at Google, they did a big defrag, completely shaking up the Hamburg engineering office. A defrag means defragmenting teams between offices. However, in my case, it just meant I needed to find a new job within Google.

'I considered a few different teams and in the end joined a hot new team. That team was working on what would eventually become Google Plus. They were well-funded and hiring.

'I worked with this team remotely for a year and then moved to California to work directly with the team, getting the L1 visa this way.

'One thing that was very different to any other company I'd worked at before, was how Google assumed competence from everyone. You would start talking to people and they assumed that you were really smart — and you would assume the same about everyone else. It made for a very nice working environment.

'I observed most companies work the opposite way. Everyone assumes everyone else is incompetent until proven otherwise. I assume this comes from hard-learned lessons on how some people are, indeed, not very competent.

'At Google, I really enjoyed this assumption of competence. I vividly remember how major a change this was. Even though I had not worked at dysfunctional places before, it made a big difference.'

4. Eleven years at Google

'I joined Google as L4. Eleven years and four promotions later, I left as L8, and I was close to an L9 promotion. This would have been a promotion on the management track to Senior Director rather than Distinguished Engineer as I didn't have the right project portfolio for an IC promotion.

'I first worked on the team responsible to inject the Google Plus button to webpages. I then joined the infrastructure team. One of the things I'm most proud of during my time at Google, is how that infrastructure team still exists and works on the same frameworks that my team built in those days.

'Internal mobility was high at Google, and moving was encouraged. When I was a manager, I also encouraged people to move around every two years or so; not because I wanted them to leave, but to make sure they knew they had this support.

'Google had lots of "internal recruitment" taking place as well. People pitched opportunities to each other, and that's how most of us moved teams.'

On promotions

'I have some thoughts on promotions, given I went through them several times.

'I echo what most people say: don't over-focus on promotion. I definitely did not focus on it. I *did* take them very seriously when the opportunity to get promoted came around. Back in the day, promotions were a lot of work to put together packages for. Thankfully, I always got promoted the first time round, and did not have to do a second round.

'All of my promotions included heavy documentation and story-telling. For my first promotion to L5, this meant putting together a long package of my contributions. Several people who did not know my work looked through these contributions and decided on the promotion.

'Google later changed the process, so at least two promotion committee members came from the domain you worked in, so they had some context of your work. They still would not know you personally, but at least you would not have to explain why your domain was important.

'Back in the day, before Google made this change, people would get feedback on their promotion saying, "What you did is great, but we don't think your project is important, so you won't get a promotion." Needless to say, this was frustrating for those people not getting promoted for that reason.'

On career progression at Google

'What I like about Google's model is that you get promoted to the job that you are already doing. This prevents the Peter Principle from happening; of getting promoted to a level at which you are out of your depth.

'With Google's promotion philosophy, you never get promoted to do anything that you are not good at. This was both something I liked when working at the company, and something I would keep not just at Google, but any place I work at.

'When it comes to what I think about career progression, I'm less certain. Career progression is a human need, but I also think people should be comfortable finding a place where they feel good. I was lucky enough to work at places which I liked.

'Too much pressure to get promoted over and over again is not always a good thing. Google made the change from expecting people to always get promoted to L5. Nowadays, Google is happy when you stay at L4 for a long time, and there's no more pressure to get promoted above this stage.'

On what tenure teaches you

Given Malte had long tenure at companies — more than 20 years of working at 3 companies — I asked what he's learned by staying for long periods of time at a single firm.

'Tenure gives you a lot of context. This context is helpful in executing. You'll have a better understanding of why things are the way they are, and what's been tried already but did not work, and why that was.

'The other thing that tenure brings is that you *actually* know how to get stuff done. There are always shortcuts to take and some process can be pushed here or there to get things done quicker. But you need to understand how the organization works pretty well, to be able to do this.

'If you come into organizations and stay for only a short time, it is very difficult to get this understanding. The other option is to be direct about just doing things the way you

think they should be done. This second approach is often less than appreciated by colleagues. It's hard to make friends with this approach.'

5. Infrastructure vs Product

'At Google, I worked on infrastructure-heavy teams, and slowly transitioned to more product-focused teams towards the end of my tenure. When I say infrastructure-heavy, I specifically mean framework-heavy work, rather than databases or networking.

'I've found that being a great product engineer and being a great framework engineer has overlaps.

'If you're a framework engineer, you need to be empathetic and think about how people will build something with that framework or API. If you're building a framework, you have to think, "what are all the products that someone would build on top of this?" Then you need to pick the solution that satisfies the most people.

'I believe you sometimes also need to get your hands dirty by writing an app using your framework, every once in a while. There's only so much you can imagine in your head without experiencing what it's like to use your framework.

'For example, one of the things we built was the stack to build web applications on top of Google's microservices. After we tried this out and deployed it, I then joined the Google Photos team to ship the initial version of Google Photos on top of this framework we had just built. I was building product features and continuously updating infrastructure to fit this use case better.

'This framework is based on the one publicly known as Wiz, but extended it to handle all frontend serving and data fetching concerns.

'As a framework engineer, joining a product team using your framework is very important to do, every now and then. I'm not saying everyone on the team needs to do it, or that it has to be for very long. However, doing it occasionally makes you a better professional.'

6. From Google to Vercel

'I almost joined Vercel a year earlier than I did. But Google offered me a new role that I found interesting, so I decided to stay in my comfort zone. I got the mission done in about a year, and thought again of what I'd like to do.

'I then decided I was ready to go all-in on a problem space I'm excited about, and joined Vercel. I felt there is a multitude of untapped opportunities that can be transformative to help people write software better. Vercel already has an incredible reach to the community, and so can impact more software engineers with my work. And helping people write better solutions is what I've been excited about in my career for the longest time.'

On Edge computing

'One of the most exciting things in tech is the move to Edge computing. Vercel, Cloudflare and other companies give the ability for clients to run code; not on the client, but very close to the client — therefore on the Edge. This area is very exciting.

'When folks first started to think about this, everyone thought it would be great. However, when you look a little closer, you realize there's lots of complexity. Deployed to the Edge, your code could be running 20 to 50 milliseconds away from the user. That might be a lot better than if it ran on the US East datacenter which is 150 milliseconds away.

'However, let's say your data is still in the eastern US. Now, if you're building a shopping application, you need to display things like the products for the page, the shopping cart count, and the shopping cart products. You're now looking at a waterfall of requests. If your database is 150 milliseconds away, you have a 300 millisecond round trip latency, which is much worse than what you can get on the Edge. So how do you program in this environment?

'**Computing on the Edge is a paradigm shift.** Also, there are missing capabilities in the Edge computing stack. For example, there's not yet a way to access an event queue on the Edge. Still, in some cases you really want data that doesn't change often,

to be close to zero milliseconds away. To have the ability to push it onto a queue would be great.

'The challenge in this field is we have to figure out how to the infrastructure well, and get the developer experience right, so people can build even better experiences and products than what they have today.

On what was surprising at Vercel

'People at Vercel are very excited about working here, and there's a real drive to ship things. I really liked working at Google and would absolutely go back, except for that I did not feel the kind of energy consistently across the company that I feel at Vercel. I'll go as far as saying that if I ever went back to Google I would try and reinstate some of this excitement, because this excitement is really important for getting things done.

'Right now, Vercel is in hypergrowth phase. They hired hundreds of people in a short amount of time. This comes with both positives and challenges.

'For example, we're still working to fully develop a promotion process for all roles. Some career paths are already in place, but more will come together.

'Vercel is conscious about tackling challenges one by one. They are not designing their internal processes based on the learnings of big tech companies of the last decade, because then you'd end up with a small startup using Google's promotion process, which isn't the right fit.'

Takeaways

To my mind, Malte has achieved professional growth on a scale most software engineers dream of. His work at Google has had a large impact; not just on Google, but also on how frontend engineers work, through his influence on Google's Core Web Vitals. Malte also got promoted to the Principal Engineer level; a position not all engineers get the opportunity to reach at the company.

Of course, everyone's journey is unique, and I hoped that by understanding Malte's path I would have new insights into what makes a truly successful career, and how you

set new goals, after you've achieved a rare level of success.

Following our conversation, I noted down these thoughts.

Successful careers at a company often build upon experience gained beforehand.

Initially, I was curious on how Malte was able to grow within Google to the Principal Engineer level. However, it turned out that what happened before Google was just as important.

For one, Malte has been coding professionally since he was 14. Also, he got involved with the global JavaScript community very early - which opened up horizons many others might have missed out on. Finally, the fact that he grew teams and was a manager before — even if he didn't like being a manager that much — very likely helped him navigate a large workplace like Google is.

Job hopping is not the only way to grow in one's career. Over 22 years, Malte worked at three companies before Vercel. His shortest stay at a company was two years, and he stayed for more than 8 years at both his first company, and at Google.

When it comes to tenure, there are no universal "rules" on which length of tenure helps careers the most. My view is that if you are working at a place that's not working for you, then there's little point in staying because there are plenty of other opportunities in tech. However, if you keep learning, growing, and pushing yourself professionally, then it might be worth staying longer.

Tenure does become more significant as you grow to Staff-and-above levels. As I write in the article, [Preparing for Promotions](#):

'Tenure becomes increasingly important as you become more senior. While it is common to see early career people change jobs often, not staying at any company beyond two years can easily become a flag for above-senior engineering and engineering manager positions.

'Staff and principal engineers and engineering managers need time to build connections, and to execute on longer-term, impactful projects. Driving this impact

is done by understanding the context, making sense of the business, and prioritizing work correctly. Learning this usually takes at least a year at most organizations.

'Tenure also brings a new, long-term perspective that leaders need to understand first-hand. By staying on the same team and at the same company for years, you see some of your early decisions come and bite you or others, years later. You understand how something no one thought much about turned into a big problem, or how doing a bit more work earlier saved a large amount of work, a year or two later.'

Adopting a new framework requires more than just a great framework. Malte started building the internal framework Wiz, which was adopted by several large Google products. However, it was not picked up just because the framework was good, but also because people like Malte understood the needs of those other teams at Google.

When the first version of Wiz was ready, Malte joined the Google Photos team to help them adopt it. During this time, he and the Wiz team rapidly iterated on the framework so it fitted the need of Google Photos. With one customer onboarded, they then followed a similarly "customer-centric" approach.

This is a good reminder that [platform teams](#) need empathy with customers and should build trust with customers just as much - if not more! - than they rely on great engineering.

Your big job opportunity in ten years' time might come from someone you know today. I asked Malte what advice he has for finding the next job, after working at a place like Google. He did not have much advice to share, as he ended up working at Vercel after meeting its CEO Guillermo Rauch, at JSConf in 2010. They had stayed in touch over the years, after meeting 5 years before Vercel was founded, and 12 years before Malte went on to join the company.

I'd suggest keeping this story in mind, especially as in-person conferences return. Who knows, perhaps you'll meet someone whom you'll try to hire for your company in a few

years' time, or maybe they'll find you with an exciting opportunity to work at one of tomorrow's success stories.

Thanks to Malte for sharing his learnings. Follow Malte [on Twitter](#) and read insightful posts on his blog called [Industrial Empathy](#). Articles he's published here include [Viral software deadlines](#) and [Designing even larger applications](#).

How did you like this issue? 🤔

[Amazing](#) • [Great](#) • [Good](#) • [OK](#) • [So-so](#)



43 Likes

4 Comments



Write a comment...



James Tucker Jun 14, 2022 ❤️ Liked by Gergely Orosz

Really enjoyed this interview! I definitely relate to Malte's comments on tenure. I wonder if we all stuck around our jobs longer there would be less arrogance in this field - personally I'm constantly being humbled by poor code or architectural decisions I made three years ago.

♡ LIKE (9) 💬 REPLY ...



Remington Jun 15, 2022 ❤️ Liked by Gergely Orosz

The sections on tenure and joining the product team while being a part of the framework team is great advice. As a senior engineer, I am starting to understand the need to stay at a company longer to get a true understanding of the different teams and acquire the tribal knowledge we hear so much about. It is also good to hear other success stories of working very closely with your "customers", whether they are internal to your company or

external. You get a lot of raw feedback that you can use to improve your system that you would not have otherwise known about.

♡ LIKE (5) 💬 REPLY ...

2 more comments...

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing